

Income Prediction Project Report

Name	Seat Number	Section No.
Youssef Ahmed Mohamed Elhady	20241701005	26
Youssef Diao Farouk	20241701018	26
Moaz Mohamed Hussien	20241700855	22
Mostafa Medhat Ahmed	20241700847	22
Mostafa Nasser Abd-Elhadi	20241700849	22
Moaz Mostafa Fathy	20241700856	29
Youssef Ahmed Zaki	20241701002	26

1. Project Description

Welcome to the documentation for the **Income Prediction Project**. This project aims to build a machine learning pipeline that predicts whether an individual earns more than \$50,000 a year based on census data (like age, occupation, and education).

2. Preprocessing Techniques

Before feeding data to an AI model, it must be cleaned and formatted.

1. Dropping & Cleansing the dataset

a. Started to load train & test files then setting the value of nulls as '?', then knowing the number of duplicated rows to drop them, then dropping 'education' column due to existing 'education num' column that represent the education.

b. Then showing the percentage of rows that have nulls because if it was high percent we should fill it with mean, median or mode but we found that the percent not high [here we use `.isnull().any(axis=1)` to search vertically then `.sum()/len(df)`], but to be sure that the nulls not biased to specific column we display the percent of nulls in each column [`.isnull().sum()/len(df)`] & we found that the nulls not biases in specific column, is distributed to columns, So we drop them.

2. Encoding Step

As our models can not understand Categorical Data we should convert it to numerical data.

a. We used One Hot Encoding Technique that make each real special value in our dataset columns separate them to Columns by main column name + value name put **1** if the row have these value **0** if not.

b. For Income & Sex these are 2 values only "Male" or "Female" = 1 or 0 also Income so we don't use One Hot Encoding there to be more efficient we use Binary Label Encoding:

- `sex_dict = {'Male':1, 'Female':0} - income_dict = {'>50K': 1, '<=50K': 0}`

3. Handling Outliers

a. In this stage we clip or remove the extremely high & extremely low values of `fnlwgt` by calculating it using IQR, This prevents wild outliers from confusing the models.

4. Feature Selection (Correlation)

a. We calculated how strongly each feature relates to the Income target. We kept only the "Top Features" that had an absolute correlation greater than 0.05. This removes noisy, unhelpful data.

5. Data splitting

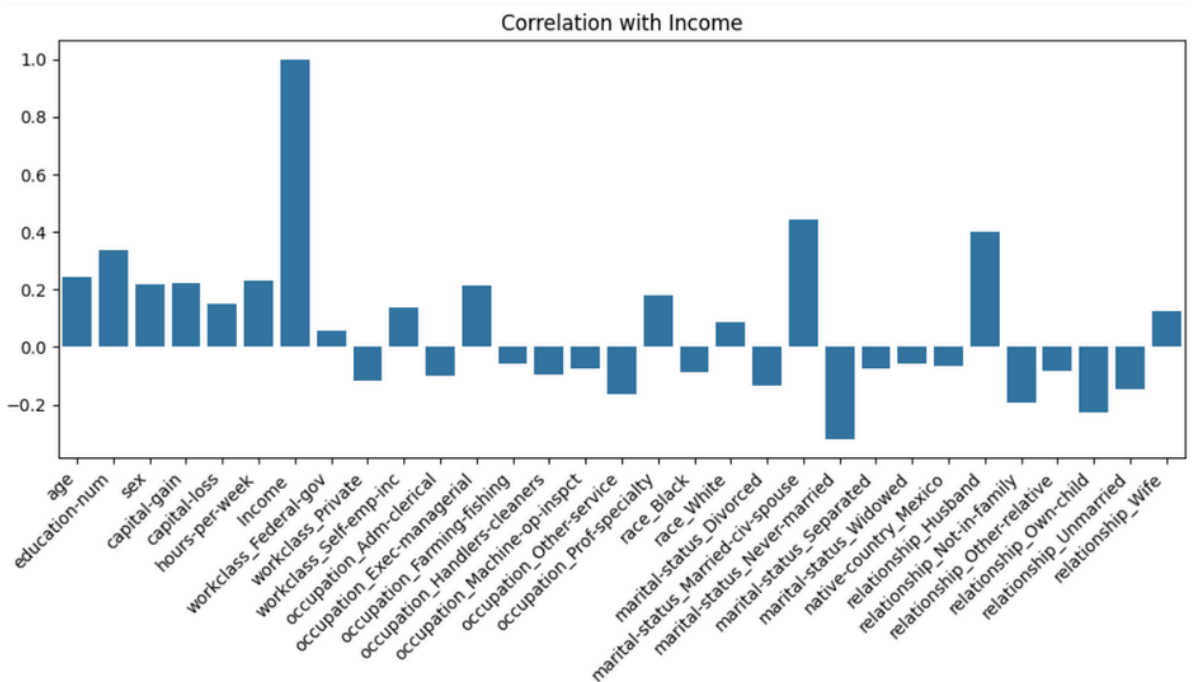
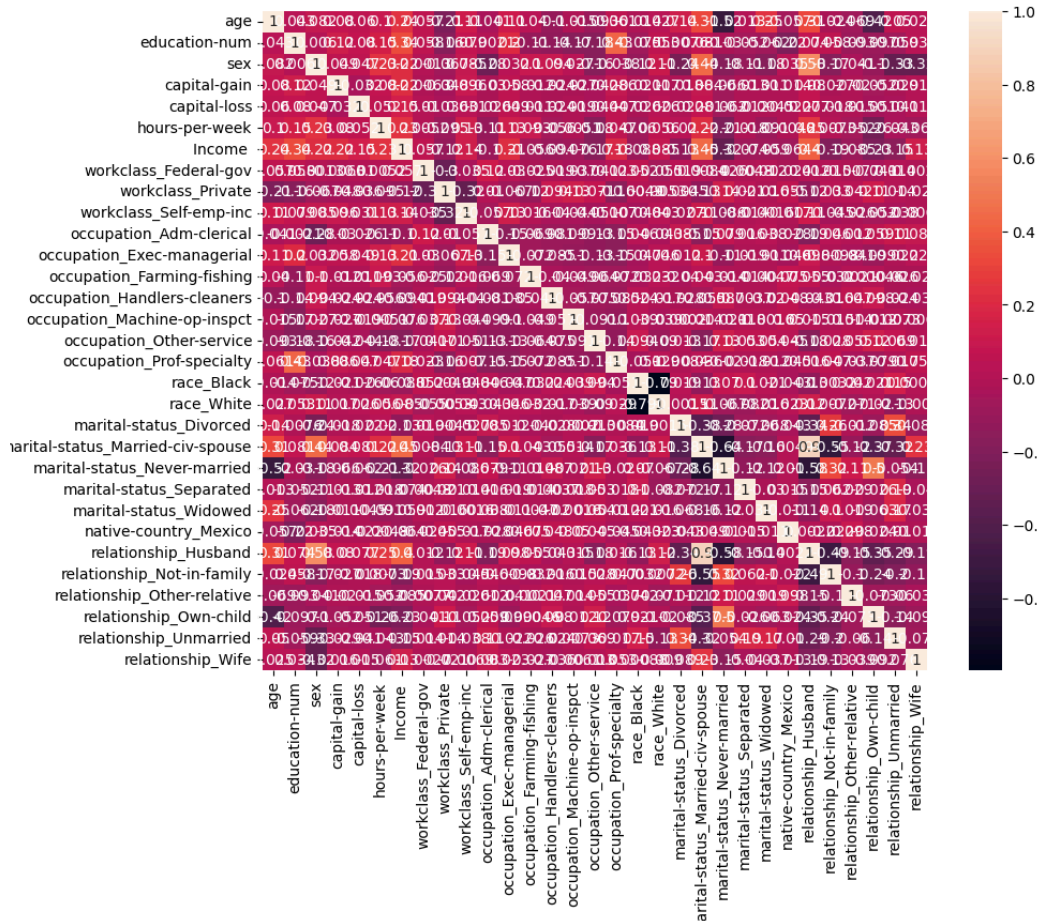
a. Split the data into `x_train` & `x_validate` then `y_train` & `y_validate`.

the train is 80% and val is 20%

6. Feature Scaling (`StandardScaler`):

a. We used `.fit()` on the training data to learn the rules, and `.transform()` on the train, validation, and test sets to apply the rules. This prevents "Data Leakage" (cheating by looking at the test test before the exam).

b. **Test Data Alignment:** We used `.reindex()` on the test dataset to ensure it had the exact same columns as the training dataset.



3. Classification Models

a. SVM

Hyperparameters:

- `C=1, gamma=0.01, kernel='rbf'`

b. Decision Tree

Hyperparameters:

- `max_depth=8, criterion='entropy', random_state=2`

c. Random Forest

Hyperparameters:

- `criterion='gini', max_depth=9, n_estimators=250`

d. Logistic Regression

Hyperparameters:

- `solver='liblinear', C=10, random_state=0`

4. Techniques used to Enhance the Result

Used in Random Forest only

a. GridSearchCV: is an automated tool in Python's Scikit-Learn library used to find the absolute best settings (hyperparameters) for a Machine Learning model, used to find the best params for RF Model by Combining different values of Hyperparameters to get the best instead of typing and guessing it manually, GridSearchCV forces the computer to test every possible combination automatically.

GridSearch Params. each one have specific meaning & effect; **1st** one was **estimator** to know on which **model** we will work, **2nd** one was **param_grid** to show the dict or **combinations** of model Hyperparams. that it will combine & test, **3rd** one was what I scoring to know what was the goal & we set it to accuracy, **4th** **cv** set to **4** & it show how many will I chop the train data to test on it the combinations to show & rate the result, **last one** we used was **n_jobs** & set it to **-1** to tell it use all device power to combine & test probabilities.

5. Conclusion

We Start our Project by uploding the train & test files then entering the 1st stage & it was Preprocessing stage the in it we detect the null value then go to clean the data & it was the 1st step in this stage, we clean the data by dropping the rows that have nulls after knowing it's percentage & dropping the duplicated rows also after we know that there percent not high & dropping them will not affect the Modeling stage not make the model biased or low the accuracy, after cleaning stage we enter Encoding step wich Our Classifaction models can't classify or train on categorical data so we use One Hot Encoding & Binary Mapping to convert categorical data into numerical ones, then we handle the outlier to clip the extremly high & low values in specific column, then we make Feature Selection on train data to show what the columns that affected our target & dropping which not affect by putting threshold 0.05; after that spliting the data into val & train, then in last stage scaling the data fitting train & applying on train, test & val; then we began to make our classification models we made: SVM, Decision Tree, Random ForestLogostic Regression.
